

Preliminares

Introducción

En los problemas de regresión actuales, hay ocasiones en las que el número de covariables supera el número de observaciones y nosotros, como estadísticos, estamos forzados a confrontar esta situación. La pregunta de cuáles predictores incluir para la regresión es difícil de contestar, y es por eso que se han desarrollado algunos algoritmos para resolver esta situación, como lo son la regresión por pasos (selección hacia delante o eliminación hacia atrás). El problema con este algoritmo es que puede descartar algunas variables que tal vez eran importantes para la regresión. A lo largo de este trabajo consideramos el modelo usual

$$y = X\beta + \varepsilon, \quad \varepsilon \sim N_p(0, \sigma^2 I), \quad (0.1)$$

donde X es una matriz de rango completo de dimensiones $n \times p$. la cual no contiene a la columna usual de 1's (notemos que el estimador $\hat{\beta}_0$ solo depende de y, X y de los demás $\hat{\beta}_j$, por lo que podemos calcularlo después). Además, supondremos que

$$\bar{y} = \sum_{i=1}^n y_i = 0, \quad \bar{x}_i = \sum_{i=1}^n x_{i,j} = 0, \quad \sum_{i=1}^j x_{i,j}^2 = 1,$$

lo cual siempre es posible realizando una transformaciones de localización y escala.

Regresión por pasos

La idea de este algoritmo es ir construyendo el modelo sucesivamente, añadiendo predictores hasta que estemos satisfechos. La correlación entre x_i y un residual $y - \hat{y}$ está dada por

$$\frac{\sum_{j=1}^n (x_{i,j} - \bar{x}_i)(y_j - \hat{y}_j - \bar{y} + \bar{\hat{y}})}{\sqrt{\sum_{j=1}^n (x_{i,j} - \bar{x}_i)^2} \sqrt{\sum_{j=1}^n (y_j - \hat{y}_j - \bar{y} + \bar{\hat{y}})^2}} = \frac{\langle x_i, y - \hat{y} \rangle}{\|y - \hat{y}\|} \propto \langle x_i, y - \hat{y} \rangle. \quad (0.2)$$

Entonces, para encontrar el predictor más correlacionado con y suponiendo que tenemos una estimación $\hat{y}$, únicamente tenemos que calcular el vector de correlaciones parciales

$$c = X^T(y - \hat{y}). \quad (0.3)$$

Luego, añadimos la variable con la correlación más alta y repetimos este proceso hasta alguna condición de paro, por ejemplo utilizando una prueba F para comparar modelos reducidos. Calcular los coeficientes de la regresión $\hat{\beta}$ en cada paso (para calcular $\hat{y}$) es computacionalmente muy costoso. Sin embargo, podemos hacerlo un poco más eficiente. Recordemos el algoritmo de ortogonalización de Gram-Schmidt. Este algoritmo ocurre también por pasos, por lo que encaja perfectamente en la regresión por pasos. Entonces, aplicamos Gram-Schmidt en la marcha y en cada paso tenemos $x_{n_1}, \dots, x_{n_r}$ y $z_{n_1}, \dots, z_{n_r}$ que son los x 's ortogonalizados. Luego, si consideramos la matriz Z con columnas $z_{n_1}, \dots, z_{n_r}$, tenemos que

$$\hat{\beta} = (Z^T Z)^{-1} Z^T y = Z^T y.$$

Como $\text{Col}(Z) = \text{Col}(X)$ y las matrices de proyección ortogonal son únicas, se tiene que $\hat{y} = X\hat{\beta} = Z\hat{\beta}$. Con esto, nuestro algoritmo queda de la siguiente manera

$$\hat{j} = \arg \max\{|c_j|\}, \quad \{z_{n_1}, \dots, z_{n_r}\} \leftarrow \{z_{n_1}, \dots, z_{n_r}, z_{\hat{j}}\}, \quad \hat{y} \leftarrow Z\hat{\beta}. \quad (0.4)$$

Regresión por etapas

Notemos que este algoritmo tiene una desventaja, si bien se llega a la solución en muy pocas iteraciones y tomamos la decisión óptima en cada paso del algoritmo, no tenemos en cuenta la optimalidad del algoritmo como un todo. Un intento de remediar este problema es la regresión por etapas. En nuestro algoritmo Ecuación 0.4, simplemente reemplazamos el término que añadimos por un $\varepsilon > 0$ suficientemente pequeño

$$\hat{j} = \arg \max\{|X^T(y - \hat{y})|\}, \quad \hat{y} \leftarrow \hat{y} + \varepsilon \cdot \text{sign}(c_{\hat{j}})x_{\hat{j}}. \quad (0.5)$$

Con esto, logramos construir la solución con incrementos más pequeños hasta que otra variable tenga mayor correlación con el residual, de modo que logramos detectar cuando este cambio ocurre (hasta un error tan pequeño como queramos). De nuevo, podemos parar el algoritmo utilizando por ejemplo una prueba F .

Tenemos el siguiente código que representa este algoritmo:

Ejemplo

Para este ejemplo consideramos una base de datos de un estudio de diabetes, donde 442 pacientes fueron medidos respecto a varias variables como edad, BMI, glucosa entre otros factores. La variable de respuesta fue una medida cuantitativa de la progresión de la enfermedad en un año. Tenemos los siguientes caminos para β .

```
library("lars")
```

```
Loaded lars 1.3
```

```

regetapas <- function(X, y, eps, iter) {

  p = ncol(X)
  beta = rep(0, p)
  incrementos = matrix(0, nrow = iter, ncol = p)

  for (i in 1:iter) {
    corrpa <- t(X) %*% (y - (X %*% beta))

    hatj = which.max(abs(corrpa))

    beta[hatj] = beta[hatj] + (eps * sign(corrpa[hatj]))

    incrementos[i, ] = beta
  }

  return(list(hatbeta = beta, camino = incrementos))
}

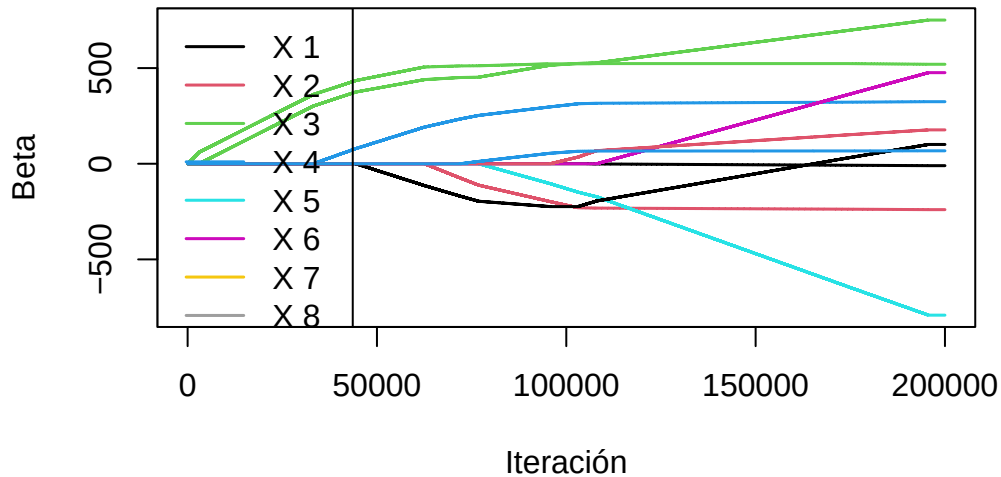
data(diabetes)
X = diabetes$x
y = diabetes$y
eps = 0.02
p = ncol(X)
iter = 200000

results = regetapas(X, y, eps, iter)

matplot(results$camino, type = "l", lty = 1, lwd = 1.5,
        xlab = "Iteración", ylab = "Beta",
        main = "Caminos de beta en la regresión por pasos")
legend("topleft", legend = paste("X", 1:p), col = 1:p, lty = 1, lwd = 1.5)

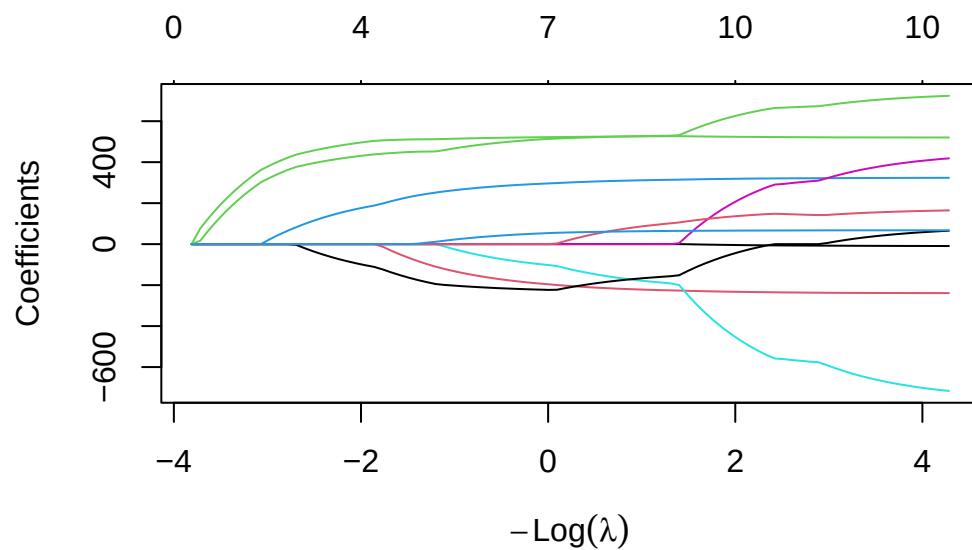
```

Camino de beta en la regresión por pasos



Ahora bien, con los mismos datos, realicemos la regresión Lasso y tenemos los siguientes caminos para β .

```
library(glmnet)
out <- glmnet(X,y)
plot(out)
```



La desventaja clara es que en este caso, el número de iteraciones necesarias para alcanzar el resultado se dispara. Ahora bien, notemos que si corremos el código de LASSO obtenemos unos caminos muy similares a los obtenidos con la regresión por etapas. Esto nos indica que podría haber una conexión entre estos dos algoritmos.